

SEMANA 1 · UNIDAD 1

Introducción al Aprendizaje Automático

 19 – 25 sep 2026
  Clase sincrónica: sábado 19/09 · tutoría 3 h + laboratorio 1.5 h

 48 h: 4.5 ACD · 18 APE · 25.5 AA

 RAI

 CG-GI

 CE-GI

Bibliografía: BB1 · BB2 · BB3 · BB5 · BC1 · BC3

MANUAL M1 · TEORÍA DE LA SEMANA

Fundamentos, proceso, métricas y fuga de datos

Lectura base de la semana 1 (2.5–3 h). Cada sección termina con un enlace **Practica esto** → que lleva al nivel exacto del ejercicio o a la animación que la ilustra.

CONTENIDO

- | | |
|---|------------------------------|
| §1 ¿Qué es el aprendizaje automático? | §6 Métricas de evaluación |
| §2 Tipos de aprendizaje | §7 Fuga de datos |
| §3 Formulación del problema | §8 Reproducibilidad |
| §4 El proceso general y la generalización | §9 Autoevaluación y lecturas |
| §5 Particiones y validación | |

§1 ¿Qué es el aprendizaje automático?

El aprendizaje automático (*machine learning*, ML) estudia programas que **mejoran su desempeño en una tarea a partir de datos**, en lugar de seguir reglas escritas a mano. La definición clásica de Tom Mitchell lo resume en tres elementos: un programa aprende de la experiencia E respecto de una tarea T y una medida de desempeño P si su desempeño en T , medido por P , mejora con E .

En la programación tradicional combinamos datos y reglas para obtener respuestas. En ML combinamos datos y *respuestas conocidas* (ejemplos) para obtener las reglas: un **modelo** $\hat{f}$ que aproxima la relación desconocida f entre las entradas x y la salida y :

$$y = f(x) + \varepsilon, \quad \hat{f} = \arg \min_{g \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(y_i, g(x_i))$$

donde $\mathcal{H}$ es la familia de modelos que consideramos (rectas, árboles, redes...), L es una **función de pérdida** que mide el error de cada predicción y ε es el ruido irreducible: la parte de y que ninguna información de x puede explicar.



Cuándo no usar ML

Si una regla simple y estable resuelve el problema (validar el dígito verificador de una cédula), si no hay datos representativos del caso de uso, o si un error no es tolerable ni explicable para quien toma la decisión. ML añade incertidumbre, mantenimiento y riesgos; debe ganarse su lugar frente a la alternativa sencilla.

§2 Tipos de aprendizaje

La diferencia entre los tipos de aprendizaje está en la **señal** que guía el aprendizaje.

TIPO	SEÑAL	TAREAS TÍPICAS	EJEMPLO DE TI
Supervisado	pares (x, y) con la respuesta conocida	regresión (y numérica), clasificación (y categórica)	detectar correos de phishing; estimar el tiempo de resolución de un ticket
No supervisado	solo x : la estructura y la similitud entre ejemplos	clustering, reducción de dimensionalidad, detección de anomalías	segmentar el tráfico de red; detectar inicios de sesión inusuales
Por refuerzo	una recompensa r por cada acción	aprender una política de decisiones secuenciales	políticas de reintento o de autoescalado de servidores

En el **aprendizaje por refuerzo** un agente observa un estado s , elige una acción a , recibe una recompensa r y pasa a un nuevo estado s' . Su objetivo es maximizar el *retorno* acumulado. Q-learning estima el valor de cada acción en cada estado y lo corrige con cada experiencia:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

El dilema central es **explorar o explotar**: probar acciones nuevas para descubrir si son mejores, o repetir la mejor conocida. La estrategia ϵ -greedy explora al azar con probabilidad ϵ . El caso más simple es el *bandido multibrazo*: un solo estado y varias acciones con recompensas desconocidas.

Practica esto → **E1.1 · Niveles 1 y 2** (regresión desde cero, K-means sobre `make_blobs` y un bandido ϵ -greedy) · **Animación A1.1**

§3 Formulación del problema

La mayoría de los proyectos de ML que fracasan no lo hacen por el algoritmo, sino por un problema mal formulado. Antes de abrir un notebook responde por escrito:

1. **Unidad de análisis.** ¿Qué es una fila? Un cliente, un correo, una sesión, un mes de un cliente. Mezclar niveles (varias filas del mismo cliente en train y test) es una fuente de fuga.
2. **Variable objetivo.** ¿Qué se predice exactamente, quién la etiqueta y con qué calidad? “Abandono” puede ser “canceló el contrato en los próximos 30 días”.
3. **Momento de predicción.** ¿Cuándo se usará el modelo y qué variables existen en ese instante? Toda variable posterior está prohibida (§7).
4. **Métrica y costo del error.** ¿Qué es peor, un falso positivo o un falso negativo, y cuánto? (§6).
5. **Baseline.** ¿Qué obtendría una regla trivial? El modelo debe superarla con claridad para justificar su costo.

§4 El proceso general y la generalización

Un proyecto de ML recorre un ciclo: **recolección** de datos, análisis exploratorio, **preprocesamiento** (imputar, escalar, codificar), **entrenamiento**, **validación** para elegir el modelo y sus hiperparámetros, **prueba** final, despliegue y monitoreo. Cuando el monitoreo detecta que los datos cambiaron, el ciclo vuelve a empezar.

Lo que importa no es el error en los datos de entrenamiento, sino el **error de generalización**: el error esperado en datos nuevos de la misma población. Un modelo demasiado simple tiene **sesgo** alto (*subajuste*: se equivoca incluso en entrenamiento). Uno demasiado flexible tiene **varianza** alta (*sobreajuste*: memoriza el ruido y falla en datos nuevos). Para la pérdida cuadrática:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(f(x) - \mathbb{E}[\hat{f}(x)])^2}_{\text{sesgo}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{varianza}} + \underbrace{\sigma^2}_{\text{ruido}}$$

El error de entrenamiento siempre baja al aumentar la complejidad; el de validación dibuja una **curva en U**. El mínimo de esa curva indica la complejidad adecuada. Las herramientas contra el

sobreajuste son más datos, regularización y modelos más simples.



Baseline: el piso que hay que superar

Un `DummyClassifier(strategy="most_frequent")` predice siempre la clase mayoritaria; un `DummyRegressor(strategy="mean")`, la media. Si el 74 % de los clientes no abandona, el baseline ya tiene 74 % de exactitud. Todo resultado se reporta junto al baseline.

Practica esto → **E1.1 · Nivel 3** (California Housing: baseline frente a modelo lineal) ·

A1.2 ciclo de vida · **A1.5 sub/sobreajuste**

§5 Particiones y validación

Para estimar el error de generalización necesitamos datos que el modelo no haya visto. El esquema estándar tiene tres papeles:

- **Entrenamiento:** para ajustar los parámetros.
- **Validación:** para comparar modelos y elegir hiperparámetros.
- **Prueba:** para la estimación final, **una sola vez**, con el modelo ya elegido. Se separa al inicio y no se toca.

Con pocos datos, un único hold-out de validación es ruidoso. La **validación cruzada K-fold** divide el entrenamiento en K partes; cada una valida una vez mientras las demás entrenan, y se reporta la media y la desviación estándar:

$$CV_K = \frac{1}{K} \sum_{k=1}^K \text{métrica}(\hat{f}^{(-k)}, \text{fold}_k)$$

- **Estratificada** (`StratifiedKFold`): cada fold conserva la proporción de clases. Es la opción por defecto en clasificación, sobre todo con desbalance.
- **Por grupos** (`GroupKFold`): cuando varias filas pertenecen a la misma entidad (cliente, paciente, equipo), todas van al mismo fold.
- **Temporal** (`TimeSeriesSplit`): cuando el orden importa, se entrena con el pasado y se valida con el período siguiente. Nunca se baraja.

Python

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=2026) # la prueba se separa primero

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2026)
res = cross_validate(modelo, X_train, y_train, cv=cv, scoring="average_precision")
print(f"{res['test_score'].mean():.3f} ± {res['test_score'].std():.3f}")
```

Practica esto → E1.3 · Nivel 2 · Animación A1.3

§6 Métricas de evaluación

Regresión

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2} \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

MAE está en las unidades de y y trata igual todos los errores; RMSE penaliza más los errores grandes; R^2 compara con predecir la media (puede ser negativo en validación).

Clasificación

Un clasificador produce un **puntaje**; un **umbral** lo convierte en decisión. Con esa decisión se arma la **matriz de confusión**: verdaderos positivos (VP), falsos negativos (FN), falsos positivos (FP) y verdaderos negativos (VN).

$$\text{precisión} = \frac{VP}{VP + FP} \quad \text{recall} = \frac{VP}{VP + FN} \quad F_1 = \frac{2 \cdot \text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$$

- **Exactitud** $(VP + VN)/n$: engañosa con desbalance. Con 1 % de fraude, “nunca es fraude” acierta 99 %.
- **Curva ROC y AUC-ROC**: la tasa de verdaderos positivos contra la de falsos positivos en todos los umbrales. AUC es la probabilidad de que un positivo al azar reciba mayor puntaje que un negativo al azar.
- **Curva precisión-recall y PR-AUC** (*average precision*): más informativa cuando los positivos son pocos, porque la precisión sí depende de la prevalencia. Su línea de azar es la prevalencia, no 0.5.

El costo del error y el umbral

La métrica y el umbral se eligen por el costo de cada error. Si un falso negativo cuesta c_{FN} y un falso positivo c_{FP} , el umbral adecuado minimiza el costo esperado:

$$C(t) = c_{FN} \cdot FN(t) + c_{FP} \cdot FP(t)$$

scikit-learn ofrece `TunedThresholdClassifierCV` para elegir el umbral por validación cruzada con una métrica o una función de costo, en lugar de usar 0.5 por defecto.

Practica esto → **E1.2 · Niveles 1 a 3**

(métricas desde cero y umbral por costo sobre defectos de software) · **Animación A1.6**

§7 Fuga de datos

Hay **fuga de datos** (*data leakage*) cuando información que no estará disponible al momento de predecir, o que pertenece al conjunto de validación o prueba, influye en el entrenamiento.

Produce estimaciones optimistas que se desploman en producción. Las formas más comunes:

1. **Preprocesar antes de partir:** escalar, imputar, seleccionar variables o codificar con estadísticas del dataset completo.
2. **Variables del futuro** (*target leakage*): datos registrados después del evento. En el Titanic, `boat` (bote asignado) y `body` (número de cuerpo recuperado) casi determinan la supervivencia, pero no existen al zarpar.
3. **Duplicados o entidades repetidas** entre entrenamiento y prueba.
4. **Decidir mirando la prueba:** elegir hiperparámetros, variables o umbrales con el conjunto de prueba.

La prevención es estructural: todo lo que *aprende de los datos* va dentro de un `Pipeline`, y el Pipeline completo se evalúa con validación cruzada. Así, en cada fold, las transformaciones se ajustan solo con los datos de entrenamiento de ese fold.

Python

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

num = Pipeline([("imputar", SimpleImputer(strategy="median")),
                 ("escalar", StandardScaler())])
cat = Pipeline([("imputar", SimpleImputer(strategy="most_frequent")),
                 ("onehot", OneHotEncoder(handle_unknown="ignore"))])
prep = ColumnTransformer([("num", num, cols_num), ("cat", cat, cols_cat)])
modelo = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
    
```



Síntoma de fuga

Resultados “demasiado buenos” (exactitud cercana a 1 en un problema difícil), una variable con importancia desproporcionada o un desempeño que cae fuertemente entre validación y producción. Ante la duda, pregunta por cada variable: *¿existiría este dato en el momento en que el modelo hace la predicción?*

Practica esto → **E1.3 · Nivel 1** (fuga sobre ruido puro) y **Nivel 4**
 (encontrar la fuga en Titanic) · **Animación A1.4**

§8 Reproducibilidad

Un resultado que nadie puede reproducir no es un resultado. La norma d del curso exige entregar un **repositorio reproducible**:

- **Entorno declarado:** `pyproject.toml` (con `uv` o `pip`) o `environment.yml` (conda), con Python 3.13 o 3.14 y versiones fijadas.
- **Semillas fijas** en toda fuente de azar: `random_state=2026` en particiones, validación cruzada y modelos.
- **README** con los pasos exactos para ejecutar de principio a fin y los resultados esperados.
- **Datos obtenidos por código** (p. ej. `fetch_openml`), nunca archivos modificados a mano; sin credenciales ni datos personales (norma e).
- **Declaración de uso de IA** generativa y de código externo, con los prompts relevantes (norma f).

Practica esto → **E0 · Entorno reproducible**

§9 Autoevaluación y lecturas

▼ 1. Un modelo de fraude tiene 99.2 % de exactitud con 0.8 % de transacciones fraudulentas. ¿Es un buen modelo?

No se puede saber: el baseline “nunca es fraude” obtiene 99.2 %. Hay que mirar recall, precisión y PR-AUC, y compararlos con el baseline.

▼ 2. ¿Por qué no se debe ajustar `StandardScaler` antes de `train_test_split`?

Porque la media y la desviación incluirían las filas de prueba: información del test influiría en el entrenamiento (fuga).

▼ 3. ¿Cuándo usarías `TimeSeriesSplit` en lugar de `StratifiedKFold`?

Cuando las filas tienen orden temporal y el modelo se usará para predecir el futuro: se entrena con el pasado y se valida con el período siguiente, sin barajar.

▼ 4. El error de entrenamiento es 0.01 y el de validación 0.40. ¿Qué pasa y qué harías?

Sobreajuste (varianza alta). Opciones: más datos, regularización, un modelo más simple o menos variables.

▼ 5. En un detector de intrusiones, ¿prefieres ROC-AUC o PR-AUC para comparar modelos?

PR-AUC: el tráfico malicioso es una fracción muy pequeña y la precisión (cuántas alertas son reales) es lo que satura o no al equipo de seguridad.

Lecturas de la semana

- **BB3** · James et al. (2023), ISLP: cap. 2 (Statistical Learning) y cap. 5 (Resampling Methods).
- **BB1** · Géron (2025): capítulo del proyecto de ML de principio a fin.
- **BB2** · Murphy (2022): cap. 1 (introducción).
- **BC1** · Russell y Norvig (2021): secciones introductorias del capítulo de aprendizaje por refuerzo.
- **BC3** · scikit-learn: Common pitfalls and recommended practices.
- **BB5** · scikit-learn User Guide: Cross-validation, Metrics and scoring y Pipelines and composite estimators.